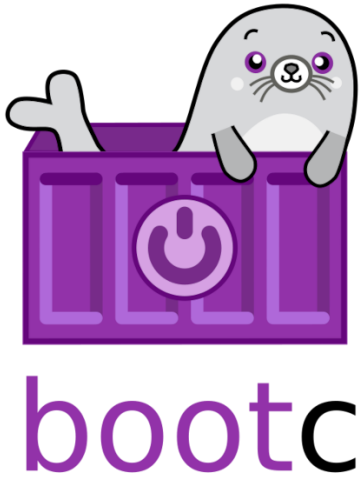
BOOTC IN THE REAL WORLD – FROM CONCEPT TO RUNNING SYSTEM

Pawel Osobinski
WITS'26

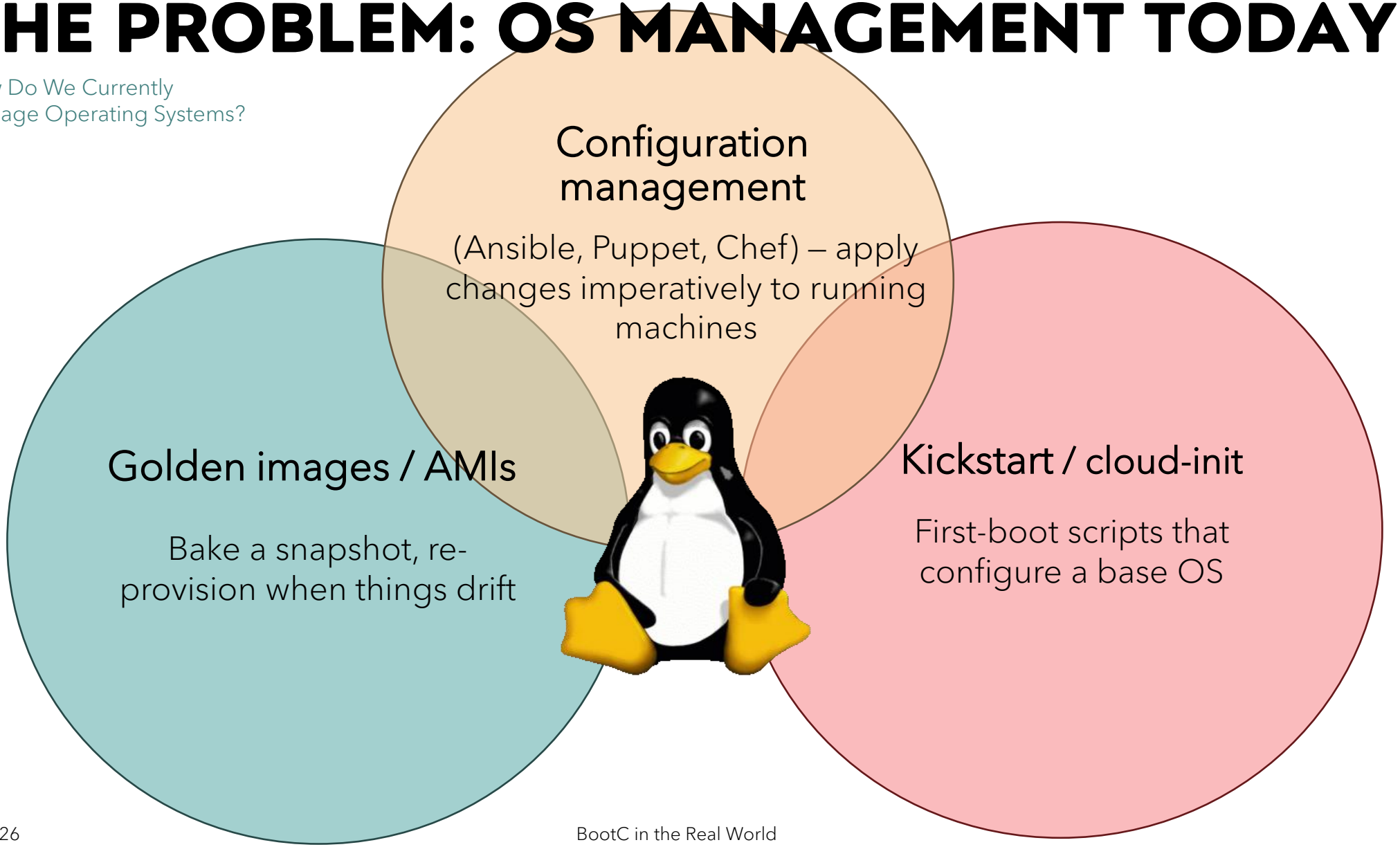
AGENDA



- What bootc is and why it exists
- The problem with managing operating systems today
- Core architecture
- Building, deploying, and updating a real system
- Fitting bootc into CI/CD
- Hands-on exercise
- Q&A

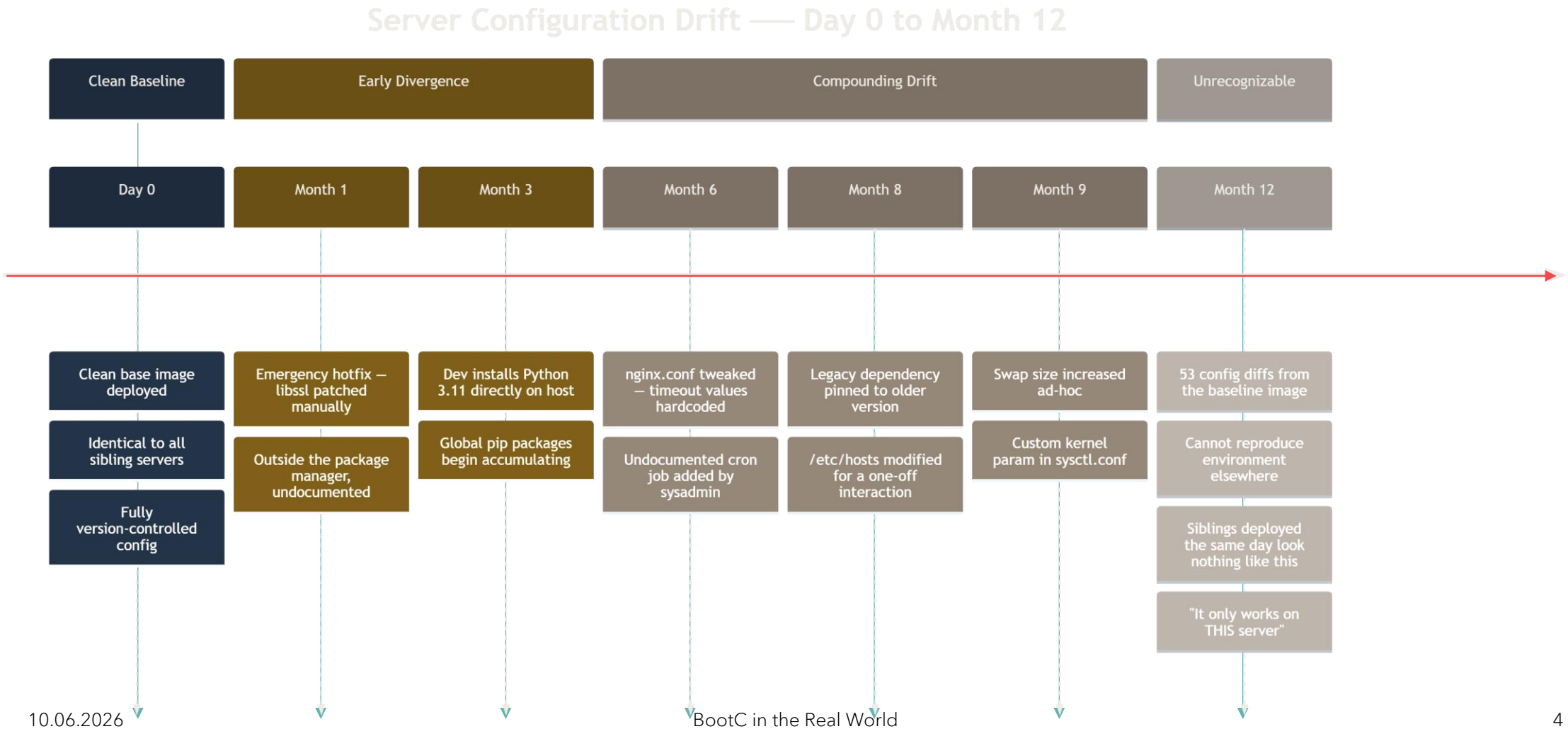
THE PROBLEM: OS MANAGEMENT TODAY

How Do We Currently
Manage Operating Systems?



THE SNOWFLAKE PROBLEM

Configuration Drift: The Silent Killer



WHAT IF WE TREATED THE OS LIKE AN APPLICATION?

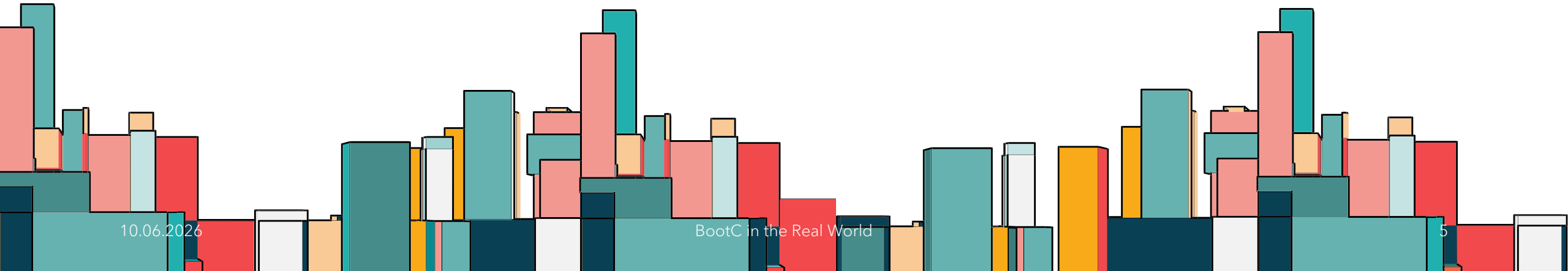
A Different Mental Model

Application today

- Defined in a Dockerfile
- Versioned and immutable
- Tested in CI
- Deployed atomically
- Rolled back trivially

OS traditionally

- Configured imperatively
- Drifts over time
- Tested manually (if at all)
- Updated in-place
- Rollback is painful or impossible



INTRODUCING BOOTC

Bootable Containers

bootc is an open-source tool and specification that allows you to define, build, and deploy a complete Linux operating system using a standard OCI container image. The system you produce is not a container running inside a host - it is the host.

Key facts:

- Open source: https://github.com/bootc-dev/bootc
- Backed by Red Hat; production-ready in RHEL 9.4 (as "RHEL image mode") and Fedora
- Uses standard podman build / docker build – no new toolchain to learn
- The base images exist today for Fedora, CentOS Stream, and RHEL

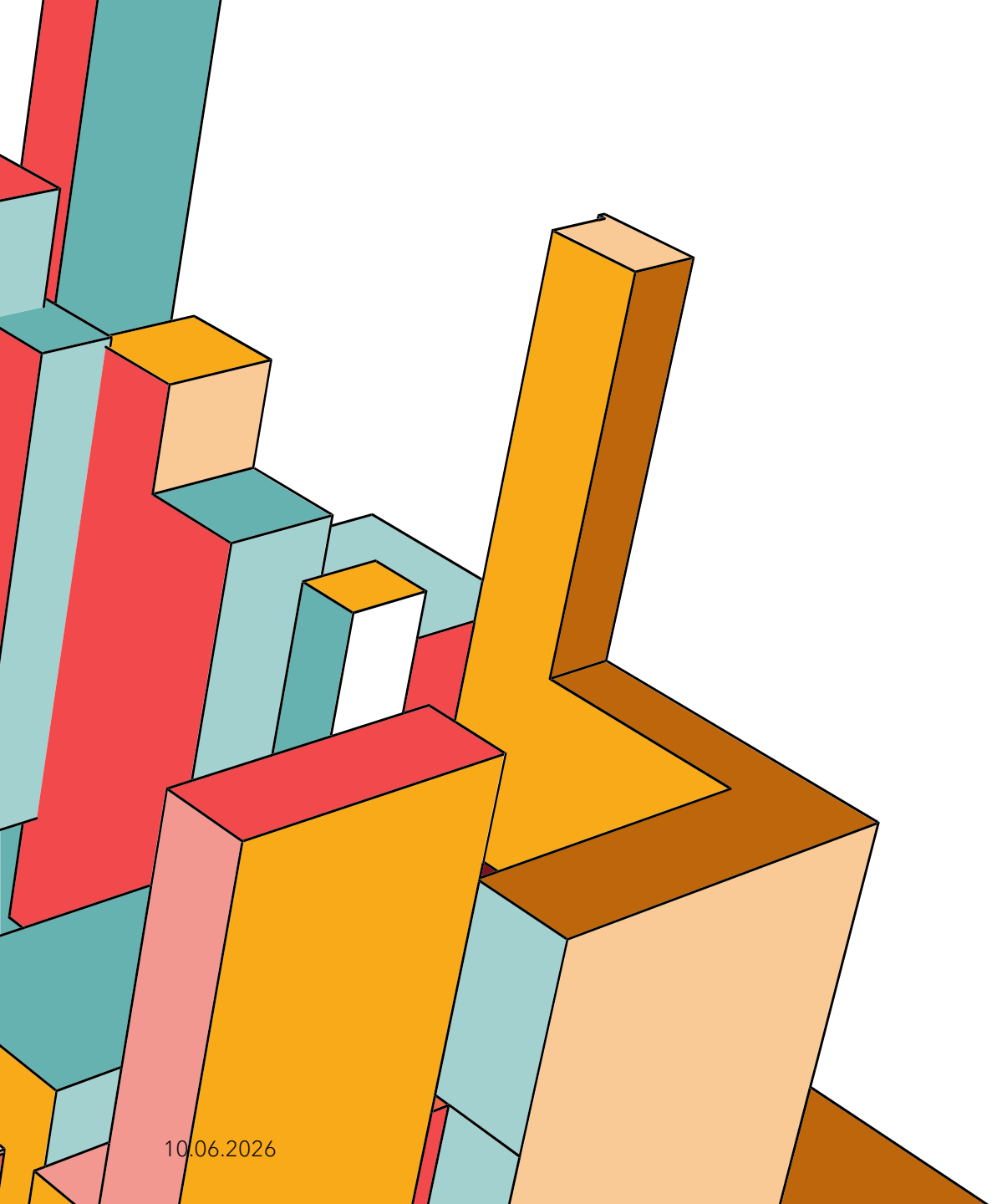
bootc

THE MENTAL MODEL SHIFT

From Configuration Target to Deployable Artifact

Before:

After:



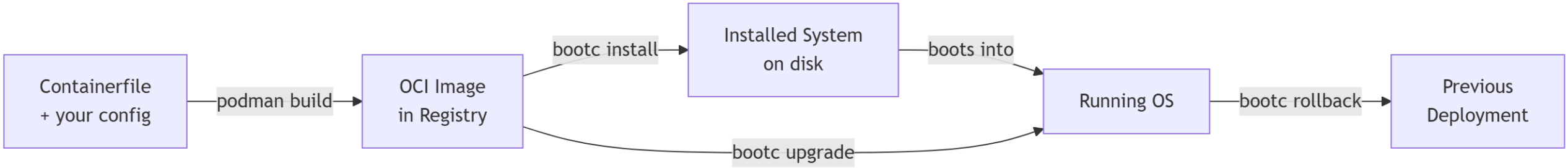
BUILDING BLOCKS

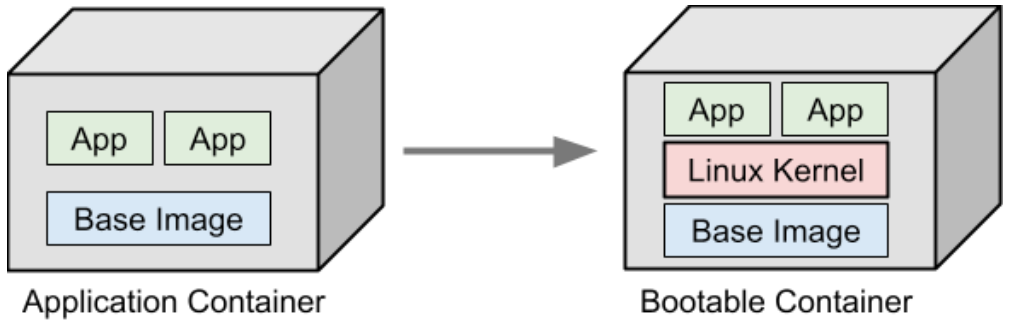
What Makes bootc Work

- **OCI Container Image is your OS definition** - built with a standard Containerfile, stored in any container registry.
- **ostree** is an immutable filesystem versioning system that works like git for OS trees. It handles the actual on-disk representation and makes atomic updates possible. You never interact with it directly.
- **bootc is the bridge** - it understands both the container image format and the ostree boot system, translating between them.
- **Container Registry** (Quay.io, GHCR, ECR, etc.) is where your OS images live, versioned and tagged just like application images.

ARCHITECTURE OVERVIEW

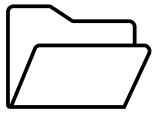
The Full Picture

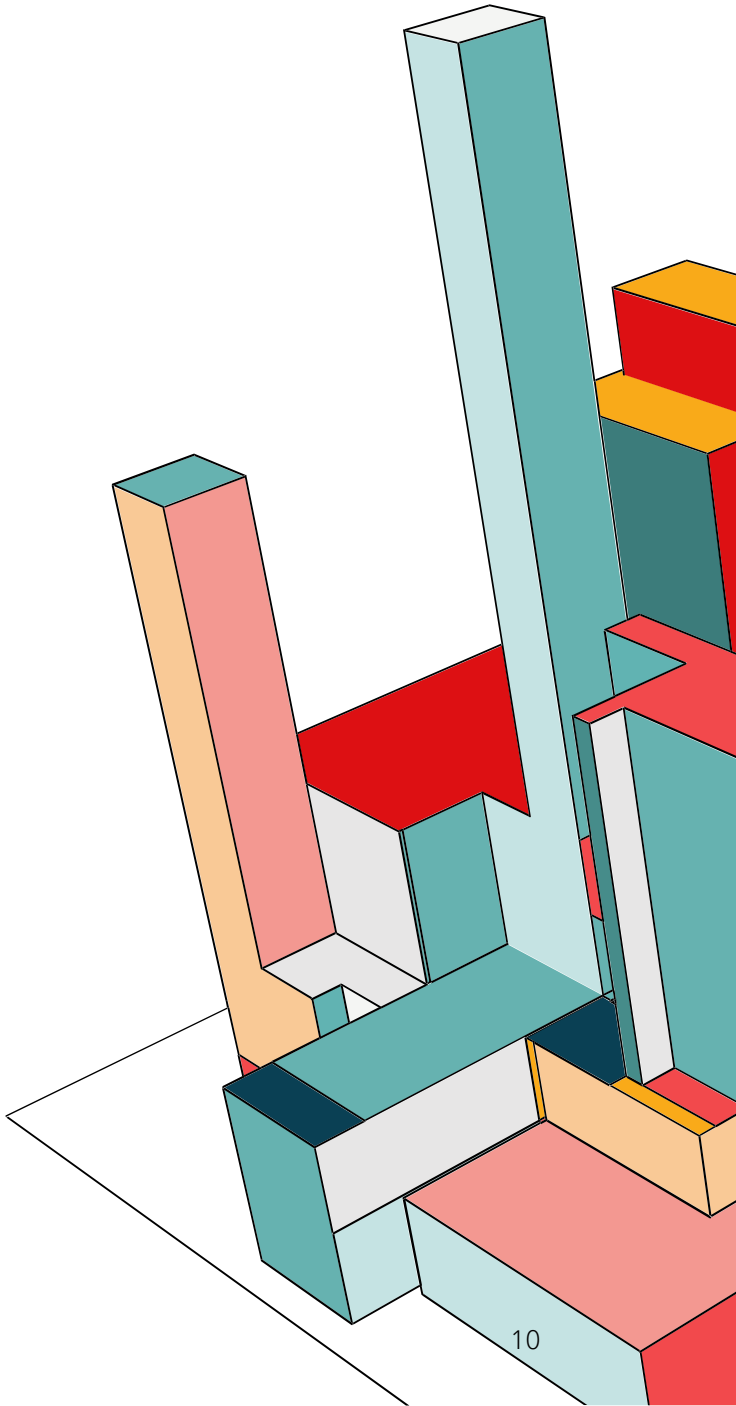




FILESYSTEM LAYOUT

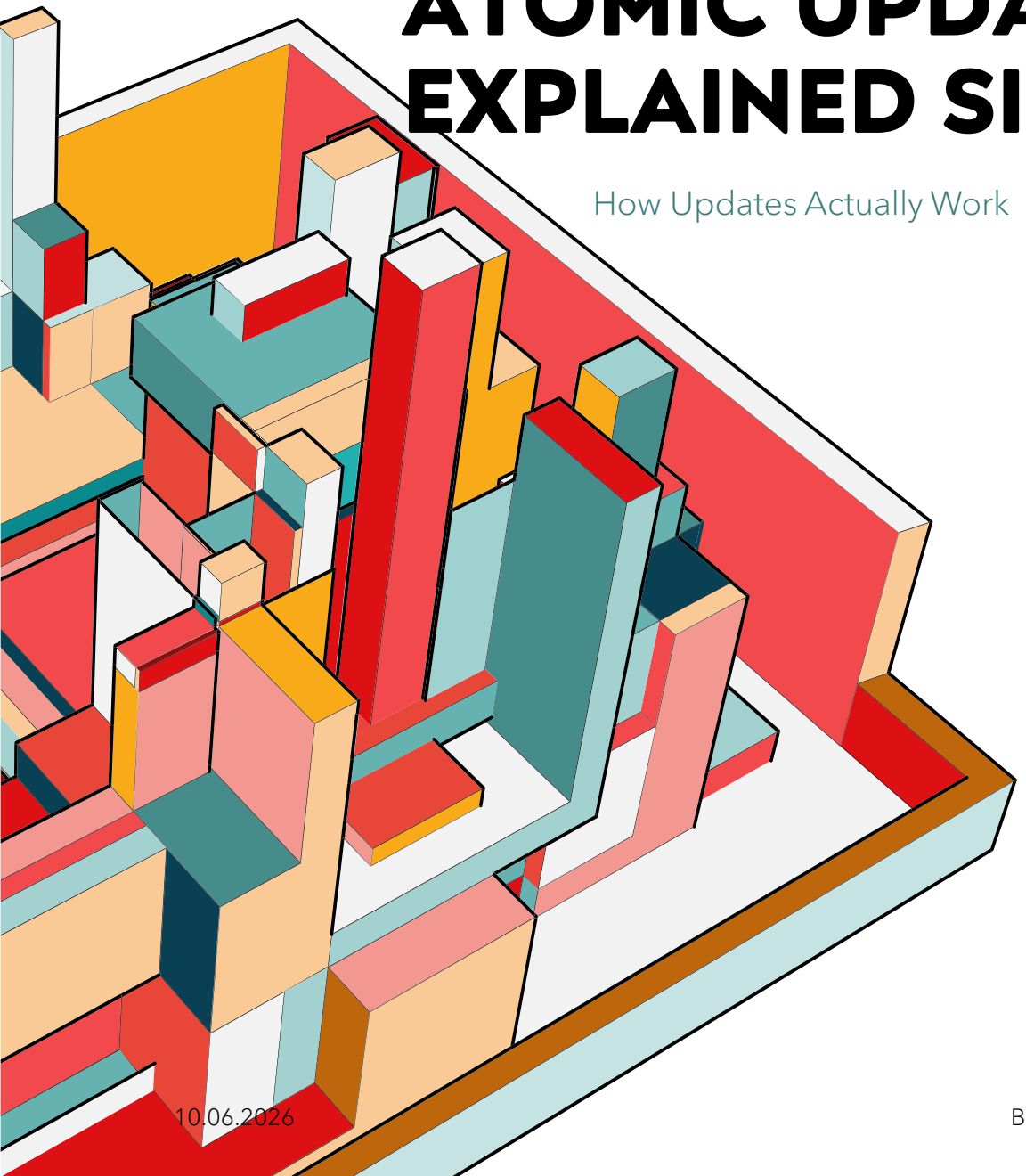
What Lives Where

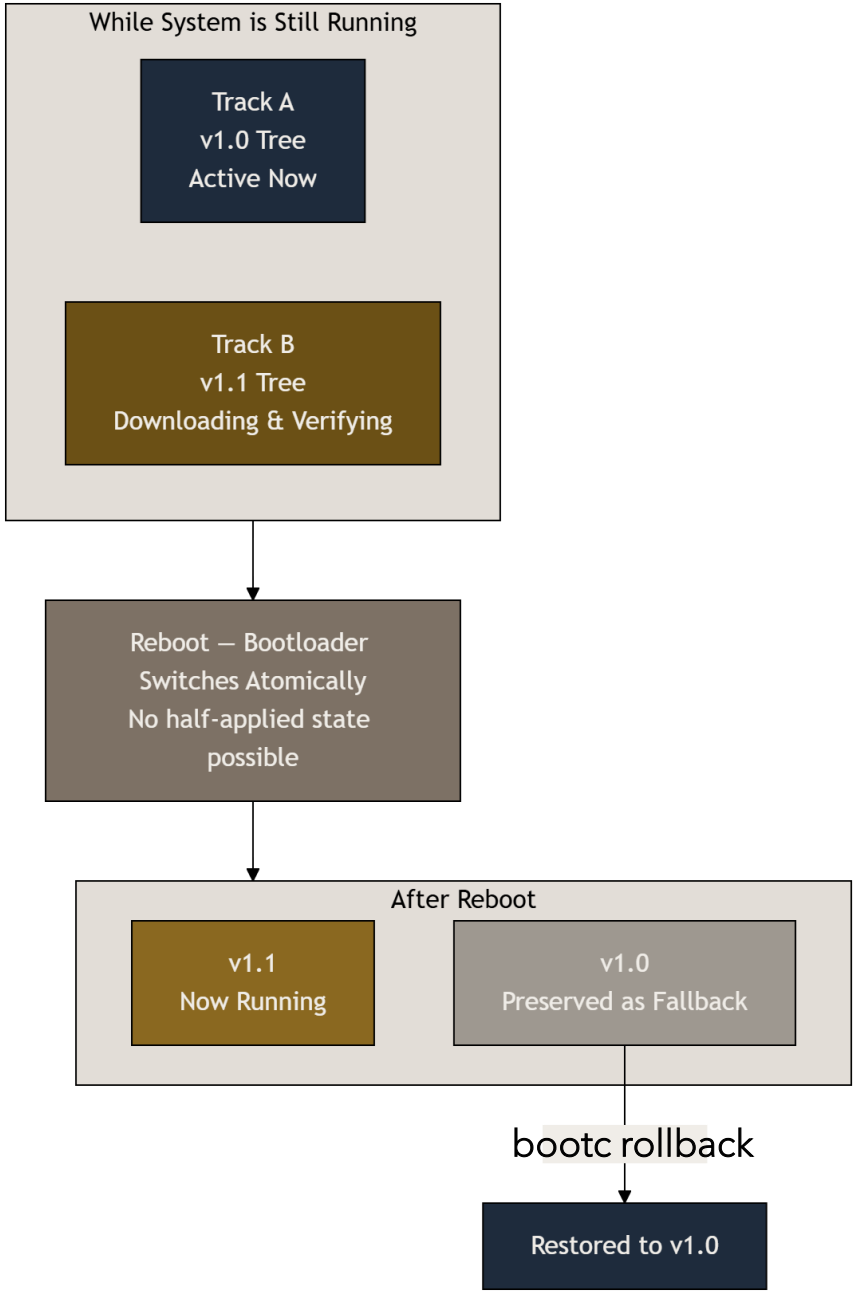
Path	Writable?		Managed by
`/usr`	Read-only		bootc (OS content)
`/etc`	Writable		Local config, persists across upgrades
`/var`	Writable		Application data, logs, databases
`/boot`	Managed		Bootloader & ostree



ATOMIC UPDATES EXPLAINED SIMPLY

How Updates Actually Work





ANATOMY OF A BOOTC CONTAINERFILE

Building Your OS – It's Just a `Containerfile`

```
FROM quay.io/centos-bootc/centos-bootc:stream10

# Install packages like any container
RUN dnf install -y nginx vim && \
    dnf clean all

# Enable systemd services
RUN systemctl enable nginx

# Drop in configuration files
COPY nginx.conf /etc/nginx/nginx.conf
COPY index.html /var/www/html/index.html
# Expose port 8080
EXPOSE 8080

# Custom motd
RUN echo "WITS BootC server!" > /etc/motd
CMD ["/sbin/init"]
```

AVAILABLE BASE IMAGES

Where to Start

The ecosystem already has ready-to-use base images for the most common enterprise Linux distributions:

- `quay.io/fedora/fedora-bootc:45` - Fedora, freely available, great for development and experimentation
- `quay.io/centos-bootc/centos-bootc:stream10` - CentOS Stream 9 and Stream 10, no subscription required
- `registry.redhat.io/rhel9/rhel-bootc:9.5` - RHEL with full support (subscription required), RHEL 10 available as well

bootc

- Debian and Ubuntu experimental images exist in the community

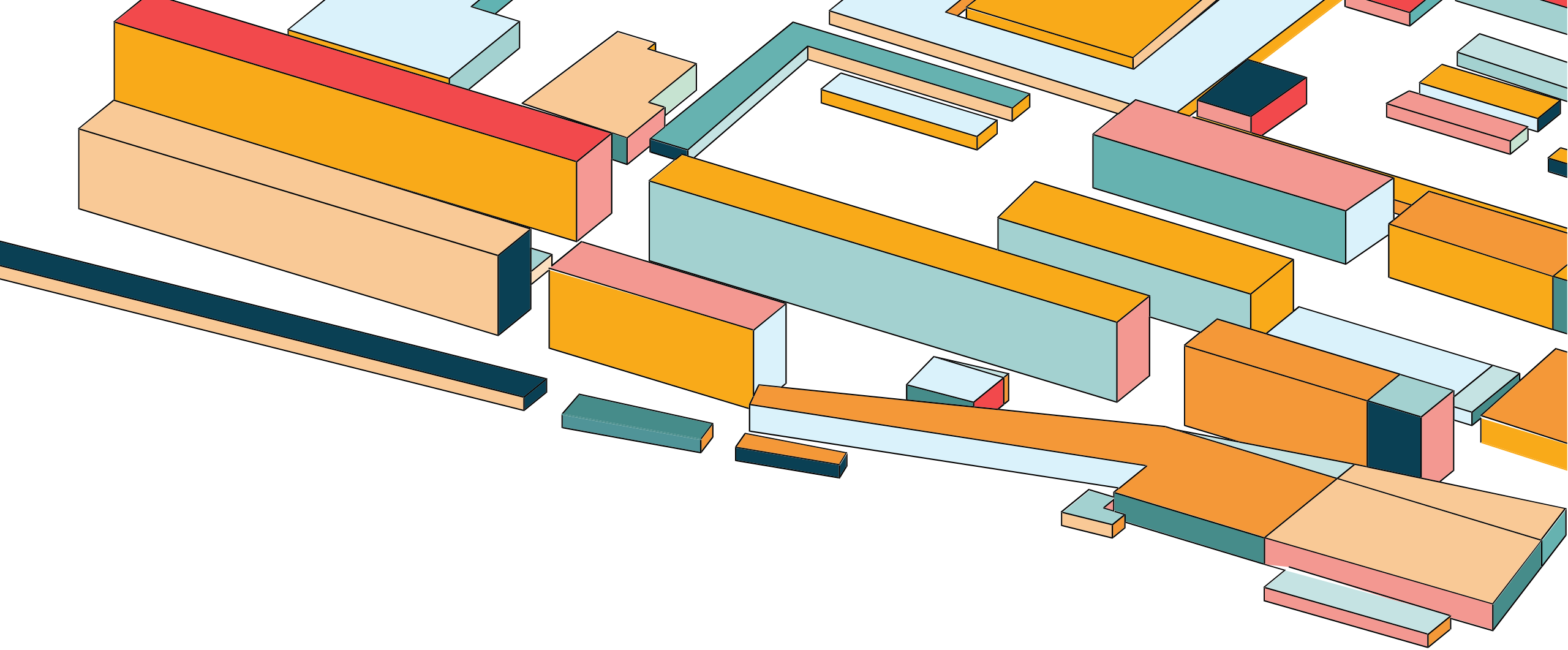
BUILDING AND PUSHING

Build Once, Deploy Everywhere

```
# Build your OS image - identical to any container build
podman build -t bootc-os:v1.0 .

# Push to a registry - your OS is now a versioned artifact
podman push myorg/bootc-os:v1.0 quay.io/myorg/bootc-os:v1.0

# Tag and push an update later
podman build -t myorg/bootc-os:v1.1 .
podman push myorg/bootc-os:v1.1 quay.io/myorg/bootc-os:v1.1
```



[DEMO] BUILDING A BOOTC IMAGE

Let's Build One

DEPLOYMENT OPTIONS

Getting the Image onto a Machine

`bootc install to-disk`

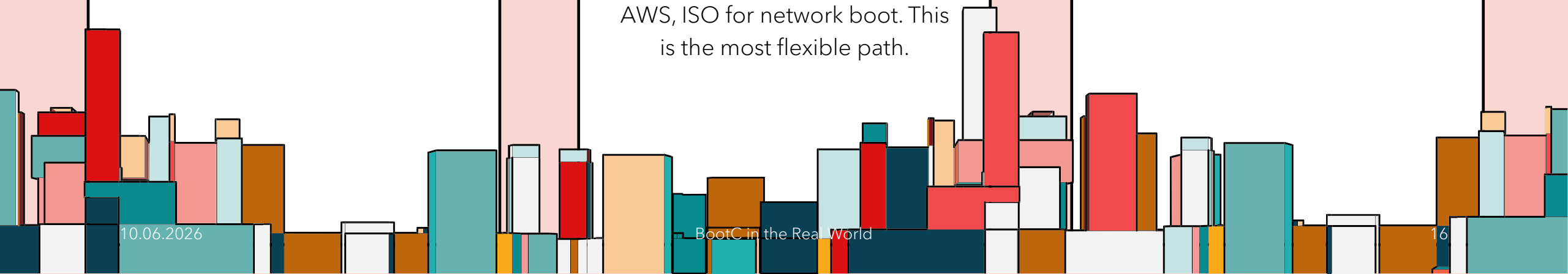
Installs directly to a block device. Use this when provisioning bare metal or a fresh VM. The system will be bootc-managed

`bootc-image-builder`

Is a container tool that takes your bootc image and converts it to a bootable disk format – QCOW2 for QEMU/KVM, raw image for bare metal, AMI for AWS, ISO for network boot. This is the most flexible path.

`bootc switch`

Converts an existing ostree-based system (such as a Fedora CoreOS machine) to be managed by your bootc image. Useful for migrations.

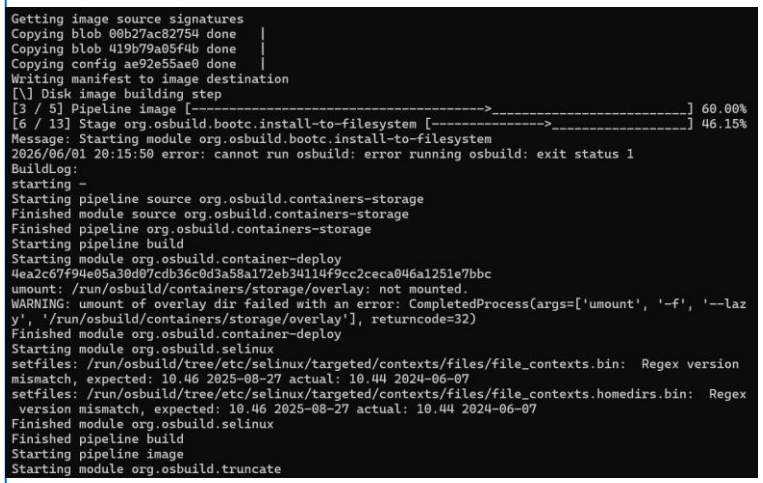


BOOTC-IMAGE-BUILDER

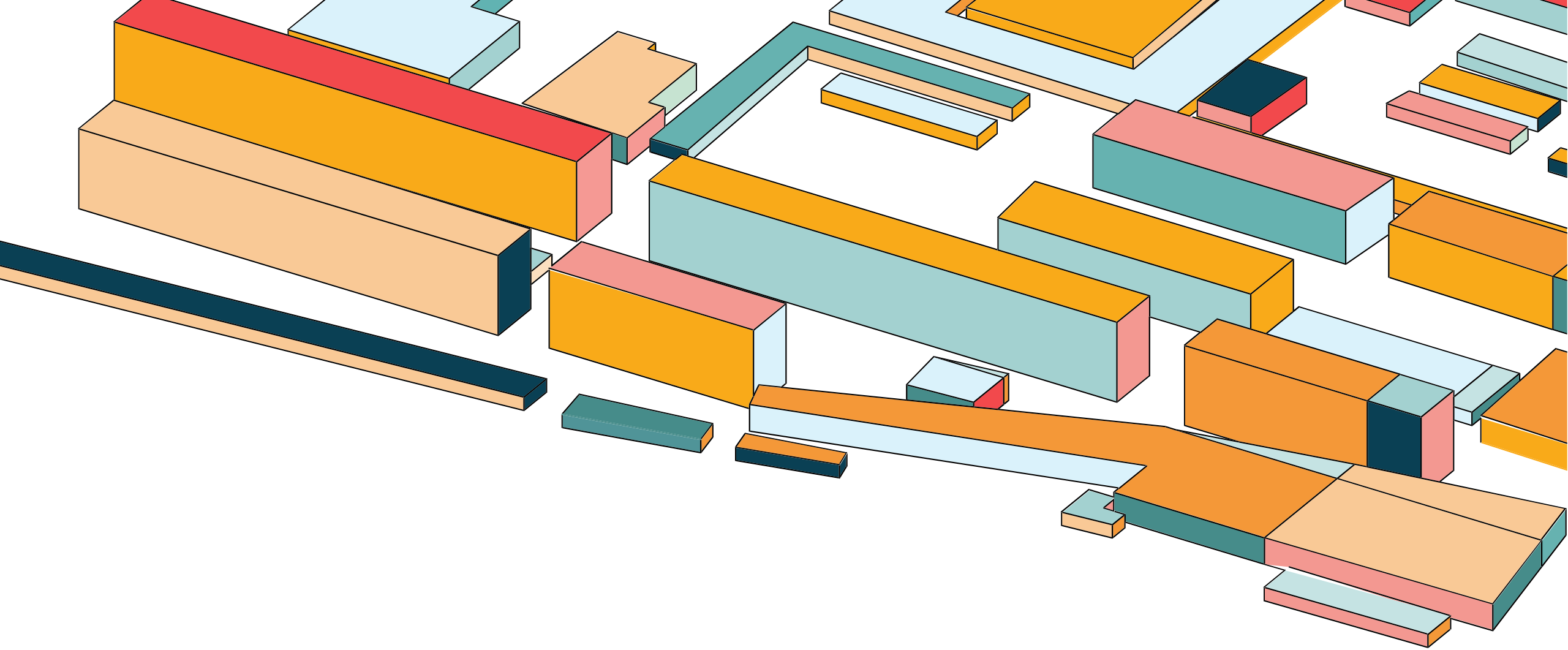
Generating Bootable Images from Your Container

```
# Generate a QCOW2 VM image from your bootc container image
podman run --rm -it --privileged \
-v ./output:/output –v \
/var/lib/containers/storage:/var/lib/containers/storage \ --security-opt label=type:unconfined_t \
quay.io/centos-bootc/bootc-image-builder:latest \
--type qcow2 localhost/bootc-os:v1.0

# Result: output/qcow2/disk.qcow2 – ready to boot in KVM/libvirt/QEMU
```



Other supported output types: `raw`, `anaconda-iso`, `vmdk`, `ami`
Command must be run in rootful podman. Extra space !



[DEMO] FIRST BOOT

The System Is Already Configured

UPGRADING THE SYSTEM

The Update Workflow

→ add a package, change a config

1. Edit Containerfile
   → new image version
2. `podman build`
   → v1.1 (latest in the same minor version) is in the registry
3. `podman push`
   → on the running system (+ reboot)
4. `bootc upgrade`

On any managed system

pulls the latest image, stages it

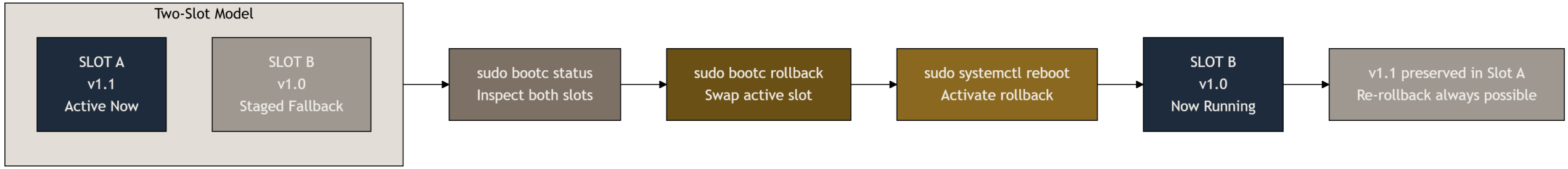
```
bootc upgrade
```

switches to new deployment on next boot

```
systemctl reboot
```

ROLLBACK AND SWITCH IMAGE

Going Back Is One Command (change as well)




```
# Check what's currently deployed
sudo bootc status
# Roll back to the previous deployment
sudo bootc rollback
# Reboot to activate the rollback
sudo systemctl reboot
```

```
# Switch to a completely different bootc image
sudo bootc switch quay.io/myorg/my-os:v2.0
# Or switch to an image from a different registry
sudo bootc switch ghcr.io/myorg/production-os:stable
# Reboot to activate image from a different registery
sudo systemctl reboot
```

NO MORE DRIFT

Immutability as a Feature

`/usr` is read-only at runtime

You cannot install packages on a running system and have them persist. Changes must flow through a new image build *

Every image is addressed by its SHA256 digest

You know exactly what is running, pinned to a specific byte-for-byte state.

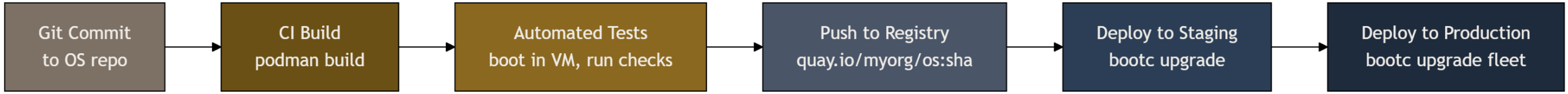
Your git history is your audit trail

Every change to the OS is a commit, reviewed, tested, and traceable - the same process you use for application code.

* Changes can be made, but all of them will be temporary

BOOTC IN A CI/CD PIPELINE

Your OS Is Just Another Artifact




```yaml
name: Build OS Image
on:
  push:
    branches: [main]

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build bootc image
        run: |
          podman build \
            -t ghcr.io/${{ github.repository }}:${{ github.sha }} \
            -t ghcr.io/${{ github.repository }}:latest \
            .

      - name: Push to registry
        run: podman push ghcr.io/${{ github.repository }}:${{ github.sha }}
```

REAL-WORLD USE CASES

Where bootc Fits Well

Edge computing and IoT

Devices in the field that must update reliably, handle interrupted downloads, and never get stuck in a half-configured state. bootc's atomic model is ideal here.

Kubernetes node OS

Worker nodes defined as a versioned image, updated through the same pipeline as the workloads running on them

Air-gapped environments

The OS is just a container image – copy it to a local registry and the rest of the workflow is unchanged

CI/CD runner infrastructure

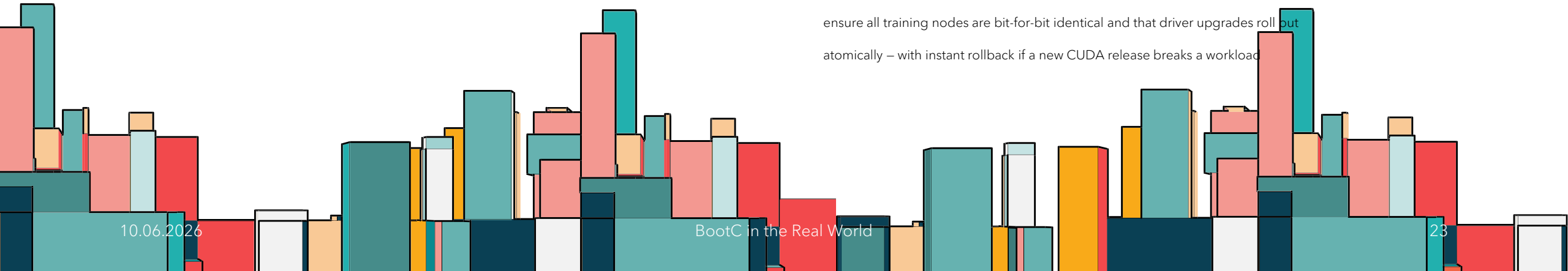
Ephemeral build agents that need to be absolutely identical across a fleet and easy to refresh when dependencies change. Update the image, recycle the runners.

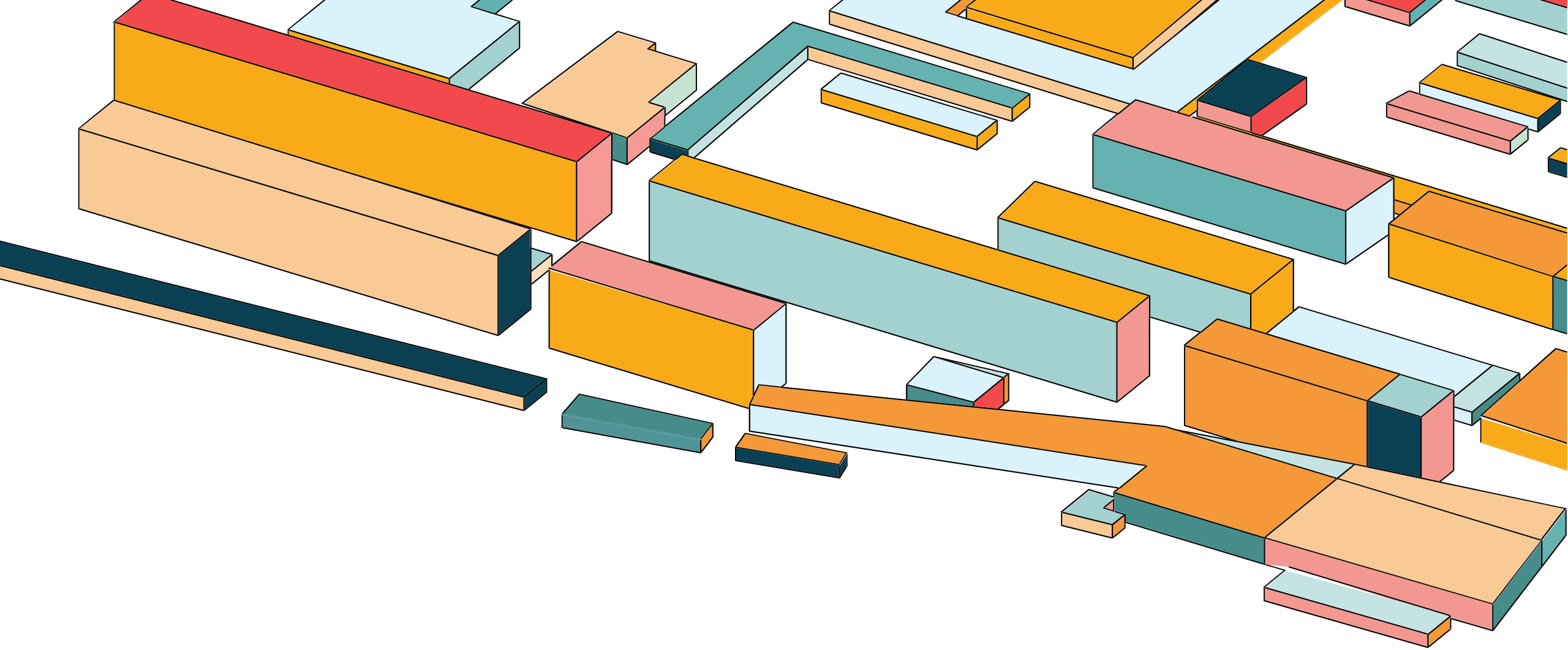
Developer workstations

A standardized, company-wide base OS that developers customize via /etc overlays while the base is kept current automatically

ML training and inference infrastructure

GPU driver stacks, CUDA versions, and kernel modules baked into a versioned image ensure all training nodes are bit-for-bit identical and that driver upgrades roll out atomically – with instant rollback if a new CUDA release breaks a workload





WORKSHOP EXERCISE: IMMUTABLE OS & IMAGE SWITCHING

HANDS-ON EXERCISE

Your Turn – 10 Minutes

Inspect the Current System & Check if `mc` is installed

```
sudo bootc status
mc --version
```

Try to Install mc the Traditional Way

```
sudo dnf install mc -y
```

Switch to the New Image (release)

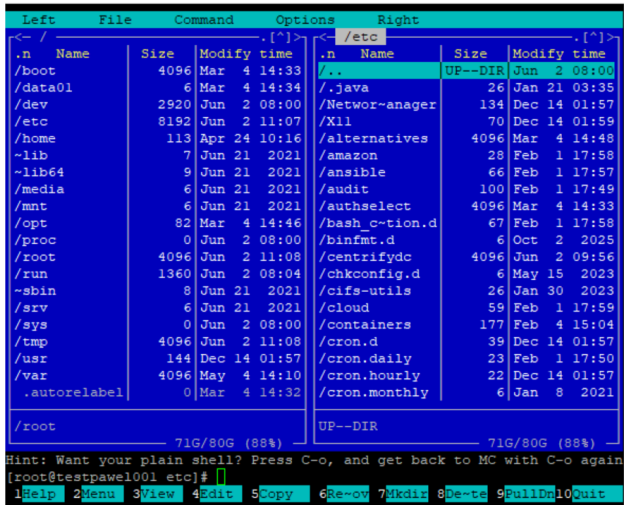
```
sudo bootc switch quay.io/pawel_osobinski72/bootc-basics:release
sudo bootc status
sudo systemctl reboot
```

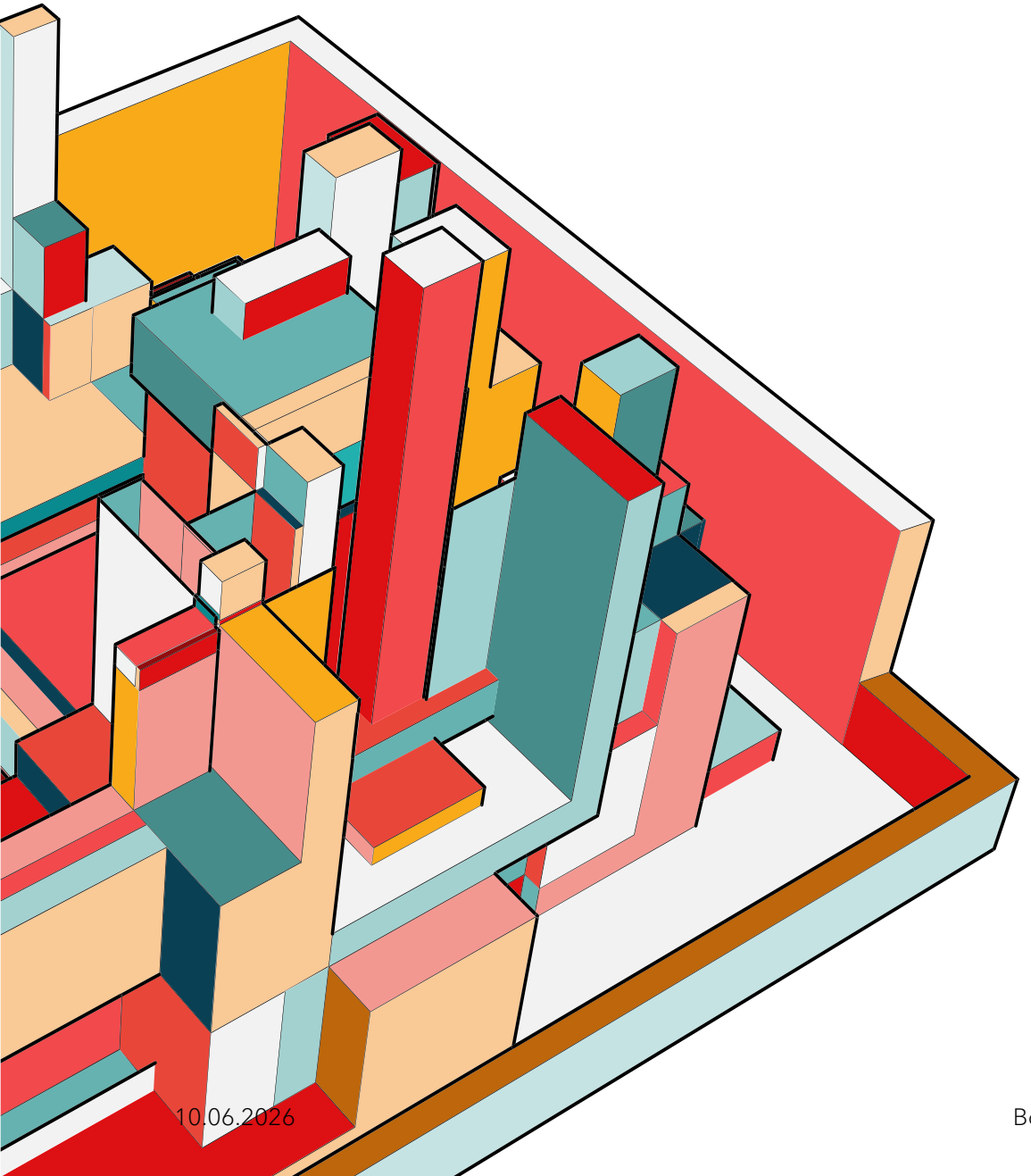
Verify the New Image

```
sudo bootc status
mc --version
```

Rollback to the Previous Image

```
sudo bootc rollback
sudo systemctl reboot
```





KEY TAKEAWAYS

The core insight is simple: define your OS the same way you define your application as a Containerfile, versioned in git, built in CI, stored in a registry. Everything else - reproducibility, atomic updates, rollbacks, audit trails - follows from that single discipline shift.
Specifically:

- bootc uses standard OCI images and standard container tooling
- Updates are atomic and always reversible
- Configuration drift becomes structurally impossible
- Your existing CI/CD pipeline handles OS delivery with minimal changes
- The learning curve is low if your team already works with containers

RESOURCES + Q&A

Official documentation and repos

Workshop repo: https://github.com/osobinp/bootc-basics

bootc docs: https://bootc.dev/bootc/

GitHub: https://github.com/bootc-dev/bootc

bootc-image-builder: https://github.com/osbuild/bootc-image-builder

RHEL image mode: red.ht/imagemode

Fedora bootc base image: https://quay.io/repository/fedora/fedora-bootc

CentOS bootc base image: https://quay.io/repository/centos-bootc/centos-bootc

THANKS !

Pawel Osobinski

pawel.osobinski@point72.com